SIMATS SCHOOL OF ENGINEERING

Approved By AICTE | IET-UK Accreditation

ASSIGNMENT 1

Natural Language Processing

Finite-State Automata, Regular Expressions and Minimum Edit Distance

Student Name	Appala Manohar
University	SIMATS (Saveetha University)
Course Code	DSA0311
Course Name	Natural Language Processing
Branch	AIML
Academic Year	2026
Assignment	Assignment 1
Register Number	192425360
Submission Date	05-11-2026

.

1. Introduction and Scope of the Assignment

This report answers the three questions of **DSA03 – Natural Language Processing, Assignment 1**. Each question is treated as a separate chapter. The first question asks for a finite-state automaton (FSA) that recognises absolute calendar date expressions. The second asks for seven regular expressions covering alphabetic strings, repetition, back-references, sentence structure and boolean word combinations. The third asks for a manual computation of the minimum edit distance between the word DRIVE and the words BRIEF and DIVERS, and a justified statement of which word is closer.

The three questions share a common theme: each defines a set of strings and asks for a precise, mechanical way of deciding membership in that set. An FSA decides membership by reading a sequence of symbols and moving between states. A regular expression describes the same kind of set declaratively. A minimum edit distance measures how far apart two strings are by counting the cheapest sequence of edit operations that turns one into the other. Because of this shared theme, the report repeatedly moves between three views of the same problem: a diagram or pattern, a table of transitions or matches, and a worked example that can be checked by hand.

Every answer in the report was checked before inclusion. The automaton was simulated on the stated examples, and its transition table was compared against an independent regular expression over all token strings of length up to six. Each regular expression was run against its matching and non-matching examples. The edit-distance dynamic-programming tables were recomputed independently by a memoised recursive implementation. Where a result depends on a modelling choice, such as the cost of a substitution or whether the empty string counts as a valid input, the choice is stated explicitly and its effect is discussed.

The report uses the following conventions. Regular expressions are written in Perl-compatible notation, which is the notation named in the assignment brief. Tables and figures are numbered by chapter. Terms are defined the first time they are used. The main academic text is in black; navy is used only for headings and table styling, as specified.

2. Question 1: Finite-State Automaton for Simple Date Expressions

Question. Design an FSA that recognises simple date expressions such as March 15, the 22nd of November and Christmas. Include all such "absolute" dates, and not "deictic" ones that are relative to the current day, such as the day before yesterday. Each edge of the graph should carry a word or a set of words, and shorthand for classes of words should be used to avoid drawing too many arcs (for example, furniture → desk, chair, table). (10 marks; CO1; BLT4)

2.1 Introduction to Finite-State Automata

A finite-state automaton is one of the simplest formal models of computation. It reads an input string one symbol at a time, changes state according to a fixed rule, and at the end of the input either accepts or rejects the string. The model has a finite number of states, which is what gives it the name. Deterministic finite automata (DFAs) are the variant used in this report. In a DFA, each state has exactly one successor for each input symbol, so the machine's behaviour on any input is fully determined.

Formally, a DFA is a five-tuple $M = (Q, \Sigma, q_0, F, \delta)$, where:

- **Q** is a finite, non-empty set of states.
- **Σ** is the input alphabet, a finite set of symbols.
- **$q_0 \in Q$** is the start (initial) state.
- **$F \subseteq Q$** is the set of accepting (final) states.
- **$\delta : Q \times \Sigma \to Q$** is the transition function, which gives the next state for each state and input symbol.

Given an input string $w = s_1 s_2 \ldots s_n$, the machine starts in q_0 and applies δ once for each symbol. The string is accepted if and only if the state reached after the last symbol belongs to F. The set of all strings accepted by M is its language, $L(M)$. A language that can be recognised by some finite automaton is called regular. Kleene's theorem states that the regular languages are exactly the languages describable by regular expressions, so the two notations are interchangeable in expressive power. Hopcroft, Motwani and Ullman (2006) give a complete treatment of this equivalence, and Jurafsky and Martin (2008) discuss its use in natural language processing.

Automata are a natural tool for text processing. Tokenisers, morphological analysers, simple date and time recognisers and many pattern-matching engines are implemented as finite automata, either directly or through compilation of regular expressions. The date recogniser in this chapter follows that tradition: it is small, its behaviour can be inspected state by state, and it can be checked exhaustively.

2.2 The Date-Recognition Problem

A date expression in running text is a short phrase that names a calendar day. Some date expressions are **absolute**, meaning their reference does not depend on when the sentence is spoken. March 15, the 22nd of November and Christmas all name a fixed day in the calendar (Christmas, as a fixed-date holiday, always falls on 25 December). Other expressions are **deictic**, meaning their reference depends on the context of utterance. Yesterday, next Monday and the day before yesterday are of this kind. The assignment asks only for absolute expressions, so deictic phrases must be rejected.

The recogniser must therefore accept a finite but reasonably broad family of absolute expressions and reject everything else. Two design decisions shape the automaton. The first is the **unit of input**. The automaton reads tokens, which are whitespace-separated words, rather than individual characters. This keeps the diagram at the level of meaning, so that a single arc can stand for “any month name”, and it avoids drawing twelve arcs for the twelve months. The second is the handling of **multi-word holidays** such as Christmas Eve or New Year's Day. Each such name is treated as one lexical unit, so the tokeniser must join its words before classification. This is a standard step in preprocessing and does not change the automaton itself.

2.3 Word Classes and Shorthand Definitions

Rather than labelling each arc with a specific word, the automaton labels arcs with **word classes**. A class is a named set of words that behave identically with respect to the date grammar. Using classes keeps the diagram small and makes the grammar easy to extend. Table 1 lists the classes used in the automaton.

Table 1: Word classes (shorthand) used as arc labels in the date automaton

Class	Definition	Examples
MONTH	The twelve English month names	January, March, November
CARD	Cardinal day numerals 1 to 31	1, 15, 22, 31
ORD	Ordinal day numerals 1st to 31st (numeral form)	1st, 2nd, 22nd, 31st
THE	The definite article used before an ordinal	the
OF	The preposition linking a day to a month	of
COMMA	The comma separating a day from a year	,
YEAR	A four-digit year numeral from 1900 to 2099	2026, 1999
HOL	Fixed-date holidays treated as single lexical units	Christmas, Christmas Eve, New Year's Day, Halloween
OTHER	Any token not in a class above (always rejected)	yesterday, banana

The CARD and ORD classes are kept separate on purpose. A cardinal numeral can follow a month (March 15) or open a day-first date (22 November), whereas an ordinal is the form required after the article in the phrase the 22nd of November. Keeping the two classes apart lets the automaton reject ungrammatical forms such as the 22 of November while still accepting the forms the question explicitly mentions.

Because the ORD class is an enumeration rather than a pattern, its irregular suffixes must be handled exactly. In English the ordinal suffix is st for numbers ending in 1, nd for numbers ending in 2 and rd for numbers ending in 3, with the exceptions 11th, 12th and 13th, which take th. Every other number takes th. Table 1a lists the complete class, so that the automaton accepts exactly the 31 ordinals the grammar intends and nothing else. The CARD class is simply the numerals 1 to 31 written without suffixes.

Table 1a: Complete enumeration of the ORD class (day numerals with ordinal suffix)

Range	Members of the ORD class
1 to 9	1st, 2nd, 3rd, 4th, 5th, 6th, 7th, 8th, 9th
10 to 19	10th, 11th, 12th, 13th, 14th, 15th, 16th, 17th, 18th, 19th
20 to 29	20th, 21st, 22nd, 23rd, 24th, 25th, 26th, 27th, 28th, 29th
30 to 31	30th, 31st

2.4 Structure of Admissible Date Expressions

Before drawing the automaton, it helps to write down the grammar of the admissible expressions. Four principal forms cover the examples in the brief and the common absolute date forms in English, with one optional suffix.

- **Form A, month first:** MONTH followed by CARD or ORD, for example March 15 or March 15th.
- **Form B, ordinal of month:** THE, then ORD, then OF, then MONTH, for example the 22nd of November.
- **Form C, day first:** CARD or ORD followed by MONTH, for example 22 November or 15th March.
- **Form D, holiday:** HOL alone, for example Christmas.

Two further patterns are accepted. A **month and year** (March 2026) is an absolute reference to a month and is accepted as a date expression. Any of the day-bearing forms may be followed by an optional year, either directly or after a comma, so March 15, 2026 and the 22nd of November 2026 are both accepted. A holiday may also be followed by a comma and a year, as in Christmas, 2026.

The complete set of accepted token sequences can be written as a single regular expression over the class symbols, where each class is abbreviated to one letter: M for MONTH, C for CARD, O for ORD, T for THE, F for OF, K for COMMA, Y for YEAR and H for HOL:

```
L  =  M(C|O)(K?Y)?  |  MY  |  TOFM(K?Y)?
    |  (C|O)M(K?Y)?  |  H(KY)?
```

Here the question mark indicates that the preceding element is optional, and the vertical bar separates alternatives. This expression is not required by the assignment; it is included because it gives a precise specification that the automaton can be checked against. Section 2.10 reports the check.

2.5 State Diagram

Figure 1 shows the state diagram. The start state q_0 is marked with an unlabelled incoming arrow. Accepting states are drawn with double circles. Every arc is labelled with the class of word that causes the transition, and where two classes share a target the labels are combined with a vertical bar. The four forms of Section 2.4 appear as four branches leaving q_0, which rejoin at the shared year suffix states q_3 and q_4.

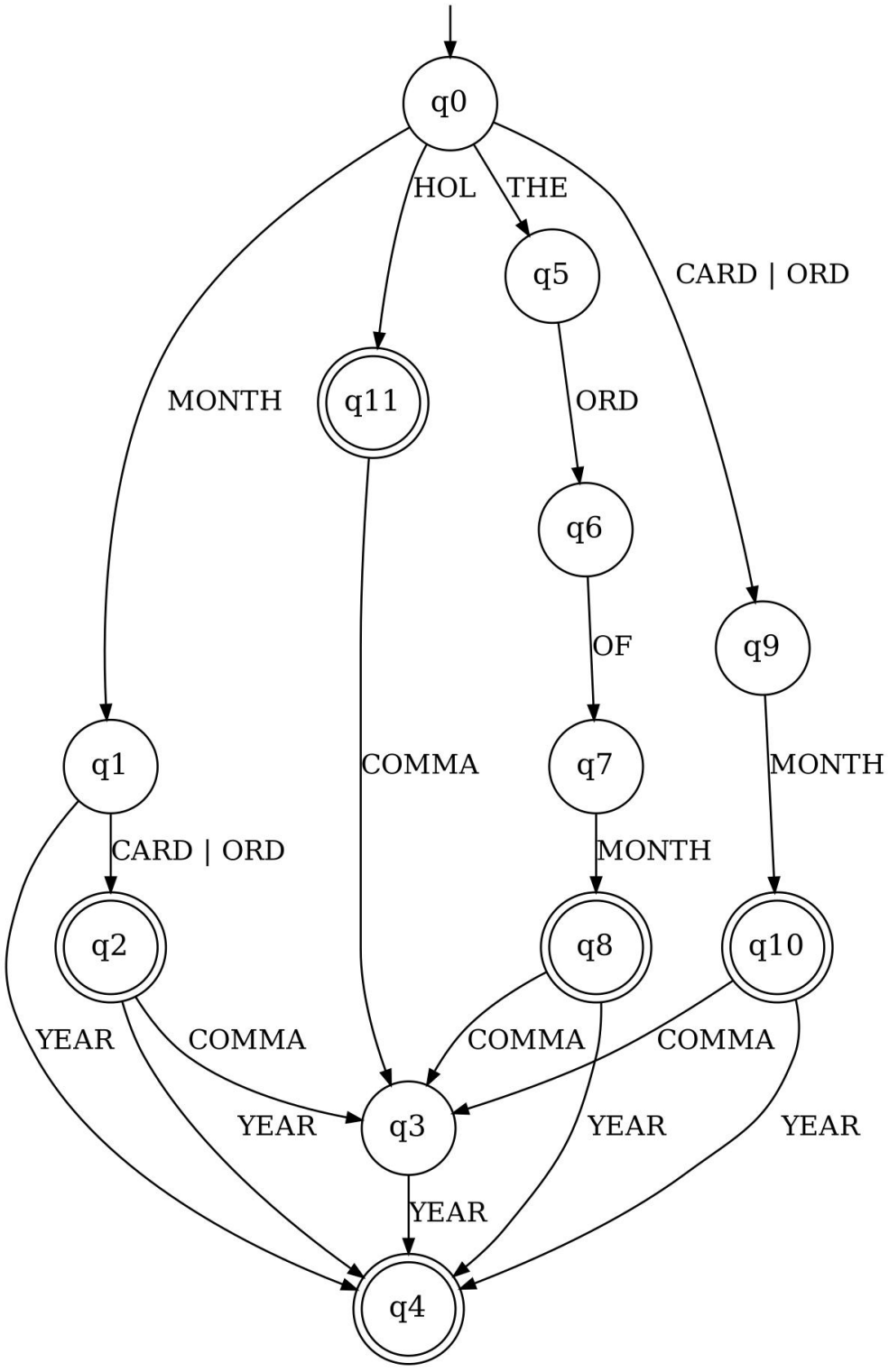


Figure 1: State diagram of the date-recognition FSA. Double circles are accepting states; q_0 is the start state.

The state labels, arc labels and transition table in Section 2.7 are plain text and can be edited directly in Word.

2.6 States, Initial State and Final States

The automaton has twelve states. Each state records how much of a date expression has been read so far. States are named q_0 to q_{11} and Table 2 describes what each one means. The initial state is q_0. The accepting states are q_2, q_4, q_8, q_{10} and q_{11}.

Table 2: States of the date automaton, their meaning and whether they are accepting

State	Meaning (input read so far)	Accepting?
q_0	Start: nothing read yet	No
q_1	A month name has been read first (Form A)	No
q_2	Month and day complete, e.g. March 15	**Yes**
q_3	A comma has been read before a year	No
q_4	A year has been read; the date is complete with a year	**Yes**
q_5	The article THE has been read (Form B)	No
q_6	THE and an ordinal have been read, e.g. the 22nd	No
q_7	The article, ordinal and OF have been read, e.g. the 22nd of	No
q_8	Complete ordinal-of-month date, e.g. the 22nd of November	**Yes**
q_9	A day has been read first (Form C), e.g. 22	No
q_{10}	Complete day-month date, e.g. 22 November	**Yes**
q_{11}	A holiday name has been read (Form D), e.g. Christmas	**Yes**

The states q_3 and q_4 are shared by all four forms. Once a comma or a year has been read, the rest of the expression is determined, so the automaton does not need separate copies of the year suffix for each branch. This is what keeps the automaton compact.

2.7 Transition Table

Table 3 gives the transition function δ. Each row is a state and each column is an input class. An entry gives the next state. A dash means that no transition is defined, so the automaton halts and rejects. Because every entry has at most one target, the automaton is deterministic.

Table 3: Transition table δ for the date automaton (– = no transition, reject)

State	MONTH	CARD	ORD	THE	OF	COMMA	YEAR	HOL
q_0 (start)	q_1	q_9	q_9	q_5	–	–	–	q_{11}
q_1	–	q_2	q_2	–	–	–	q_4	–
q_2 (F)	–	–	–	–	–	q_3	q_4	–
q_3	–	–	–	–	–	–	q_4	–
q_4 (F)	–	–	–	–	–	–	–	–
q_5	–	–	q_6	–	–	–	–	–
q_6	–	–	–	–	q_7	–	–	–
q_7	q_8	–	–	–	–	–	–	–
q_8 (F)	–	–	–	–	–	q_3	q_4	–
q_9	q_{10}	–	–	–	–	–	–	–
q_{10} (F)	–	–	–	–	–	q_3	q_4	–
q_{11} (F)	–	–	–	–	–	q_3	–	–

Reading the table row by row reproduces the structure of Section 2.4. Row q_0 has four live entries, one for each form: MONTH leads to q_1 (Form A), THE leads to q_5 (Form B), CARD or ORD leads to q_9 (Form C) and HOL leads to q_{11} (Form D). Rows q_2, q_8 and q_{10} show that a complete date may be followed by a comma and a year, or directly by a year. Row q_4 is empty because no further input can change an accepted date with a year.

2.8 Step-by-Step Recognition Examples

This section traces the automaton on representative inputs. Each step lists the current state, the token read, its class, and the next state. The traces are written so that they can be followed by hand using only Table 3.

Example 1: March 15 (accepted)

Table 4: Trace of March 15

Step	Current state	Token	Class	Next state
1	q_0	March	MONTH	q_1
2	q_1	15	CARD	q_2
End	q_2 (accepting)	–	–	**ACCEPT**

Example 2: the 22nd of November (accepted)

Table 5: Trace of the 22nd of November

Step	Current state	Token	Class	Next state
1	q_0	the	THE	q_5
2	q_5	22nd	ORD	q_6
3	q_6	of	OF	q_7
4	q_7	November	MONTH	q_8
End	q_8 (accepting)	–	–	**ACCEPT**

Example 3: Christmas (accepted)

Table 6: Trace of Christmas

Step	Current state	Token	Class	Next state
1	q_0	Christmas	HOL	q_{11}
End	q_{11} (accepting)	–	–	**ACCEPT**

Example 4: March 15, 2026 (accepted, with year)

Table 7: Trace of March 15, 2026

Step	Current state	Token	Class	Next state
1	q_0	March	MONTH	q_1
2	q_1	15	CARD	q_2
3	q_2	,	COMMA	q_3
4	q_3	2026	YEAR	q_4
End	q_4 (accepting)	–	–	**ACCEPT**

Example 5: the 22nd November (rejected)

This input is rejected because Form B requires the preposition of between the ordinal and the month. After q_6, the only defined transition is OF, so the MONTH token has no arc to follow. The automaton halts in q_6, which is not accepting. The example shows that rejection is caused by a missing transition, not only by a non-accepting final state.

Table 8: Trace of the 22nd November (halts)

Step	Current state	Token	Class	Next state
1	q_0	the	THE	q_5
2	q_5	22nd	ORD	q_6
3	q_6	November	MONTH	**no arc: REJECT**

Example 6: March 15 November (rejected)

After March 15 the automaton is in q_2. A second month name has no arc from q_2, so the input is rejected. A date may not contain two months, and the automaton encodes this by having no transition for MONTH in that state.

Example 7: Christmas, 2026 (accepted, holiday with year)

- Step 1: q_0 reads Christmas (HOL) and moves to q_{11}, which is accepting.
- Step 2: q_{11} reads the comma (COMMA) and moves to q_3, which is not accepting.
- Step 3: q_3 reads 2026 (YEAR) and moves to q_4, which is accepting. The input is accepted.

Example 8: 15th March 2026 (accepted, day-first with year)

- Step 1: q_0 reads 15th (ORD) and moves to q_9, which is not accepting.
- Step 2: q_9 reads March (MONTH) and moves to q_{10}, which is accepting.
- Step 3: q_{10} reads 2026 (YEAR) and moves to q_4, which is accepting. The input is accepted.

Examples 7 and 8 show that the year suffix is reached by the same shared state from very different forms. Example 7 arrives at q_3 through the comma arc from the holiday state, and Example 8 arrives at q_4 directly from the day-month state, so each form reuses the year arcs it needs without duplicating them.

2.9 Accepted and Rejected Date Expressions

Table 9 lists accepted expressions, the class sequence each one produces and its accepting state. Table 10 lists rejected expressions with the reason for each rejection. All entries were produced by running the automaton on the strings shown.

Table 9: Accepted date expressions

Expression	Class sequence	Path	Accepting state
March 15	MONTH CARD	q_0 q_1 q_2	q_2
March 15th	MONTH ORD	q_0 q_1 q_2	q_2
March 15, 2026	MONTH CARD COMMA YEAR	q_0 q_1 q_2 q_3 q_4	q_4
the 22nd of November	THE ORD OF MONTH	q_0 q_5 q_6 q_7 q_8	q_8
the 22nd of November, 2026	THE ORD OF MONTH COMMA YEAR	q_0 q_5 q_6 q_7 q_8 q_3 q_4	q_4
22 November	CARD MONTH	q_0 q_9 q_{10}	q_{10}
15th March 2026	ORD MONTH YEAR	q_0 q_9 q_{10} q_4	q_4
March 2026	MONTH YEAR	q_0 q_1 q_4	q_4
Christmas	HOL	q_0 q_{11}	q_{11}
Christmas, 2026	HOL COMMA YEAR	q_0 q_{11} q_3 q_4	q_4
New Year's Day	HOL	q_0 q_{11}	q_{11}

Table 10: Rejected expressions and the reason for rejection

Expression	Reason for rejection
March	Ends in q_1, which is not accepting: a month alone does not name a day.
15	Ends in q_9, which is not accepting: a number alone is not a date.
the 22nd November	No arc from q_6 on MONTH; the preposition OF is required.
the 32nd of March	32nd is not in the ORD class, which covers 1st to 31st only.
March 15 November	No arc from q_2 on MONTH; two months cannot occur in one date.
March 15 of 2026	No arc from q_2 on OF; the year must follow a comma or directly.
March, 2026	No arc from q_1 on COMMA; the month-year form has no comma.
the Christmas	No arc from q_5 on HOL; the article applies only to ordinals.
yesterday	Token belongs to OTHER; no arc from q_0. Deictic expression.
the day before yesterday	Token the leads to q_5, then day is OTHER; no arc. Deictic expression.

2.10 Explanation of How the Automaton Works

The automaton reads the tokens of a candidate expression from left to right. The state records the **shape** of the date read so far, and each new token either extends that shape or is rejected. The start state branches into four families according to the first token: a month name, the article the, a day numeral or ordinal, or a holiday name. Once the family is known, the

automaton only needs to remember which part of that family's template has been matched, and this is exactly what the states encode.

The year suffix is handled by two shared states. The comma state q_3 accepts only a year, and the year state q_4 is accepting and has no outgoing arcs. Any complete date can therefore be extended with an optional year, and the year does not have to be spelled out separately for each form. The states q_2, q_8, q_{10} and q_{11} are accepting because each is a complete date on its own, while q_1, q_5, q_6, q_7, q_9 and q_3 are not, because each still requires further input.

The automaton is deterministic, so the outcome for any input is unique. There is no need for backtracking or for a search over alternatives. Rejection happens in one of two ways: either an input token has no arc from the current state, which halts the automaton at once, or every token is consumed but the final state is not accepting. Both cases are visible in the traces of Section 2.8.

The correctness of the automaton was checked in two ways. First, the traces in Section 2.8 and the lists in Section 2.9 were produced by simulating the transition table. Second, the transition table was compared, state by state, with the token-class regular expression of Section 2.4. The comparison covered every string of token classes of length one to six over the eight input classes, which is 299,592 strings in total, and the two descriptions agreed on every one of them. Because the regular expression was written from the grammar of the date forms rather than from the automaton, agreement between the two gives good evidence that the transition table implements the intended grammar.

2.11 Limitations and Possible Extensions

The automaton recognises a deliberately bounded set of expressions. It accepts only the day numerals 1 to 31, the twelve English month names and the holiday names listed in Table 1. A real system would need to extend several of these classes. Abbreviated months such as Jan. or Dec. could be added to MONTH with the same arcs. Fixed-date holidays from other calendars could be added to HOL. Years outside 1900 to 2099 could be admitted by changing the YEAR class.

The automaton also rejects some forms that many writers consider acceptable. The phrase the 22nd November, without of, is rejected because the assignment's example uses of and the grammar is kept strict. Such a form could be admitted by adding an arc from q_6 on MONTH to q_8, at the cost of one extra transition. Numeric dates such as 15/03/2026 are outside the

scope of the assignment, which asks only for word-based expressions, and would require a different tokeniser.

Finally, the automaton is purely syntactic. It recognises the form of a date expression but does not check that the day exists in the given month, so 31 February would be accepted. A semantic check would be a separate step applied after recognition, using a calendar table, and is not part of the finite-state recogniser.

2.12 Arc Economy of the Shorthand

The word classes have a measurable effect on the size of the diagram. The automaton has 20 labelled transitions, which are listed in Table 3. If every individual word were drawn as its own arc, the diagram would need far more: the 12 month names, 31 cardinals, 31 ordinals, 7 holidays and 200 years in the range 1900 to 2099 would each need a separate arc at every point where their class appears. Table 9a compares the two counts for each class.

Table 9a: Arcs needed with and without word-class shorthand

Class	Transitions in the automaton	Words in class	Arcs needed without shorthand
MONTH	3	12	36
CARD	2	31	62
ORD	3	31	93
THE	1	1	1
OF	1	1	1
COMMA	4	1	4
YEAR	5	200	1,000
HOL	1	7	7
Total	**20**		**1,204**

The total falls from 1,204 arcs to 20 labelled transitions, a reduction of more than sixty-fold. The gain is largest for YEAR, because a single class replaces 200 arcs at each of its five uses. The figure also makes a design point. The year class is defined by a range rather than by a list, so the choice of 1900 to 2099 is made in one place and the structure of the automaton is unchanged by it. The same is true of the holiday class: adding a holiday changes one set and no state.

2.13 A Non-deterministic Alternative

A non-deterministic finite automaton (NFA) differs from the DFA in two ways. From one state it may have several arcs with the same label, and it may have ε-transitions, which consume no input. An NFA accepts a string if at least one of its possible paths ends in an accepting state. Every NFA can be converted into an equivalent DFA by the subset construction, which records the set of NFA states reachable after each prefix. Every regular expression can be converted into an NFA by Thompson's construction, which is the usual route from the grammar of Section 2.4 to an automaton.

An NFA for the date language would be built directly from the four forms. A start state would branch into four sequences, one per form, and the sequences would meet at shared year states. Such an NFA could have fewer states, because it would not need separate states to record each partial year, provided the year suffix were shared. It would, however, contain choices that have to be resolved at run time. The DFA was preferred for three reasons. Each step has exactly one possible successor, so a trace is unique and can be checked by hand. The machine halts at the first token with no arc, so rejection is immediate. And, as the next paragraph explains, the first token already determines the branch, so the nondeterminism that an NFA would introduce has little to resolve.

The first token determines the family of the expression. A month name starts Form A or the month-year form. The article the starts Form B. A numeral or ordinal starts Form C. A holiday name starts Form D. Because these four first-token sets are disjoint, the NFA obtained from the grammar would make no real choice at its first step. Its later choices are also limited: after a day, a month must follow in Form C, and after the ordinal in Form B the preposition must follow. The subset construction applied to such an NFA would therefore produce a DFA of about the same size as the one in Figure 1. This is an argument from the structure of the grammar rather than a computed result, but it explains why the direct design is natural.

2.14 Implementation

The automaton can be implemented as a table lookup. Appendix A gives a short Python program that stores the transition function δ as a nested dictionary, classifies each token by testing it against the word classes of Table 1 and Table 1a, and follows the arcs. A token with no arc halts the run and rejects it. The program was run on the examples of Sections 2.8 and 2.9 and reproduced the same accept and reject decisions as the hand traces. Because the

transition table is data rather than code, extending the grammar with a new class or a new holiday requires changing the table and not the control logic.

3. Question 2: Regular Expressions

Question. Write regular expressions for the following languages. You may use either Perl notation or the minimal “algebraic” notation of Section 2.3 of the course text, but you must state which one you are using. By “word”, the brief means an alphabetic string separated from other words by white space, any relevant punctuation, line breaks and so forth. Each part (a) to (g) is answered below. (10 marks; CO1; BLT4)

3.1 Notation and Conventions

This answer uses **Perl-compatible regular expression notation**, as the brief permits. Table 11 summarises the constructs used in the seven answers. Perl notation is chosen because it supports anchors, character classes, back-references and lookahead, all of which are needed here. The algebraic notation can express parts (a), (b) and (d) directly. Part (c) needs a back-reference, which the algebraic notation does not provide, and part (f) can be written in it only by listing both orders of the two words.

Table 11: Perl regular-expression constructs used in this chapter

Construct	Meaning
`^` and `$`	Start and end of the string (or of a line, if the m flag is used). Anchor the whole string.
`[A-Za-z]`	A character class: any one ASCII letter from A to Z or a to z.
`[0-9]`, `[+-]`	A digit; or an optional sign character (+ or -).
`+`, `*`, `?`	One or more; zero or more; zero or one (optional) of the preceding element.
`.`	Any single character except a newline.
`\b`	A word boundary: a position between a word character (letter, digit or underscore) and a non-word character, or the edge of the string.
`\W`	Any non-word character, i.e. white space or punctuation.
`( … )`, `(?: … )`	A capturing group (numbered and stored) and a non-capturing group.
`\1`	A back-reference: the text matched by the first capturing group.
`(?= … )`	A zero-width positive lookahead: the enclosed pattern must match at this position, but no characters are consumed.

Two policies apply throughout. The first concerns **punctuation**. Words are treated as maximal runs of letters, so an apostrophe or hyphen counts as a separator unless the answer says otherwise, and punctuation is permitted at the places where the brief's definition of a word allows it. The second concerns **anchoring**. Where a question asks for strings as a whole, the expression is anchored with ^ and $. Where a question asks about words in running text, the expression uses \b so that it cannot match inside a longer word.

Every regular expression in this chapter was run against its examples in Python's re module, whose syntax agrees with Perl for the constructs used here. Matching results are reported as the matched text or the captured group. The examples in each part were chosen to test the boundary cases that the question is designed to exercise.

3.2 Part (a): The Set of All Alphabetic Strings

Question. The set of all alphabetic strings.

Answer

```
^[A-Za-z]+$
```

Explanation of symbols

Symbol	Explanation
`^`	The string must begin here, so no characters may precede the word.
`[A-Za-z]`	Each character must be an ASCII letter. Digits, spaces and punctuation are excluded.
`+`	At least one letter is required, so the empty string is excluded.
`$`	The string must end immediately after the last letter, so no trailing characters are allowed.

Matching examples

String	Result
`Humbert`	Matches: a whole string of letters.
`bug`	Matches: a short alphabetic string.
`a`	Matches: a single letter is a valid alphabetic string.
`The`	Matches: capital letters are included in [A-Za-z].

Non-matching examples

String	Reason rejected
`123`	Digits are not letters.
`C3PO`	Contains digits; the whole string must be letters.
`don't`	The apostrophe is punctuation, not a letter.
`big bug`	Contains a space; two words are not one alphabetic string.
(empty string)	Contains no letters, so the + quantifier (one or more) cannot be satisfied.

Punctuation and word boundaries

The anchored form above defines a set of strings. To find words inside running text, the anchors are removed and boundaries are used: `\b[A-Za-z]+\b`. In running text, punctuation separates words, so in don't the expression finds don and t as two separate alphabetic words. This follows the brief's definition, in which an apostrophe is punctuation. The boundary \b asserts that a letter run begins and ends at a transition between letters and non-letters, which prevents a match from starting in the middle of a longer token.

Algebraic equivalent

In the minimal algebraic notation, with | for disjunction and * for repetition, the same set is written as (A|B|…|Z|a|b|…|z)(A|B|…|Z|a|b|…|z)*. The form is longer because the algebraic notation has no character classes, so every letter must be listed. It also has no anchors, so it describes the set of strings but cannot express where a match starts and ends within a line. This is the main reason for preferring the Perl form when the task is searching text.

Common errors

- Writing [a-z] instead of [A-Za-z]. This excludes capitalised words such as Humbert and The, which are alphabetic strings and belong to the set.
- Omitting the anchors. Without ^ and $, the pattern finds letters inside strings such as C3PO, which is not an alphabetic string as a whole.
- Writing [A-Za-z]* instead of [A-Za-z]+. The star also admits the empty string, which contains no word.

3.3 Part (b): Lowercase Alphabetic Strings Ending in b

Question. The set of all lowercase alphabetic strings ending in a b.

Answer

```
^[a-z]*b$
```

Explanation of symbols

Symbol	Explanation
`^`	Anchors the start of the string.
`[a-z]*`	Zero or more lowercase letters. The star allows the string to be just b.
`b`	A literal b, which must be the final character.
`$`	Anchors the end of the string, so the b must be last.

Matching examples

String	Result
`b`	Matches: zero letters precede the final b.
`cab`	Matches: the prefix ca is lowercase letters, and the string ends in b.
`crab`	Matches: the prefix cra is lowercase letters, and the string ends in b.
`aaab`	Matches: repeated lowercase letters followed by a final b.

Non-matching examples

String	Reason rejected
`bat`	Ends in t, not b.
`Cab`	Begins with an uppercase letter; the class is lowercase only.
`cab2`	Contains a digit, and does not end in b.
`ba b`	Contains a space, which is not a lowercase letter.
`abc`	Ends in c, not b.

Punctuation and word boundaries

The anchored expression treats the whole string as the unit. In running text, where a word may be followed by a full stop or comma, the boundary form is `\b[a-z]*b\b`. The boundary after b means that cab. is found, because the full stop is a non-word character, but cabbage is not, because the b in cabbage is followed by a letter. The brief does not ask for case-insensitive matching, so uppercase letters are excluded by design; adding the i flag would also accept Cab.

Algebraic equivalent

The algebraic form is (a|b|…|z)*b, that is, any string of lowercase letters followed by a final b. The Perl expression ^[a-z]*b$ is the same set, with anchors added so that it applies to whole strings. The b is a literal in both notations, so no special treatment is needed.

Common errors

- Writing [a-z]+b. This rejects the one-letter string b, which contains zero letters before its final b. The star is needed, not the plus.
- Writing .*b. The dot matches uppercase letters, digits and spaces, so Cab and ba b would be accepted, although the brief asks for lowercase alphabetic strings only.
- Omitting the final $. Without it, the pattern ^[a-z]*b matches the prefix of bat, because the b is found at the start, and so wrongly accepts bat as a string ending in b.

3.4 Part (c): Two Consecutive Repeated Words

Question. The set of all strings with two consecutive repeated words, such as Humbert Humbert and the the, but not the bug or the big bug.

Answer

```
\b([A-Za-z]+)\W+\1\b
```

Explanation of symbols

Symbol	Explanation
`\b`	A word boundary before the first word, so the match starts at a word's first letter.
`([A-Za-z]+)`	Capturing group 1: the first word, stored so that it can be referred to later.
`\W+`	The separator: one or more non-word characters, which covers white space and punctuation.
`\1`	Back-reference: the second word must be identical to the text captured in group 1.
`\b`	A word boundary after the second word, so the repeated word is a whole word.

Matching examples

String	Result
`Humbert Humbert`	Matches; group 1 captures Humbert.
`the the`	Matches; group 1 captures the.
`no, no`	Matches; the comma and space form the separator, so punctuation is handled.
`bye. bye!`	Matches; a full stop and space separate the repeated word.

Non-matching examples

String	Reason rejected
`the bug`	The two words differ.
`the big bug`	No consecutive repetition: big is between the and bug.
`the thethe`	The second word thethe is not the; the \b after the first the fails inside the longer word.
`the them`	The second word them is not identical; the boundary after the first the is missing.

Punctuation and word boundaries

The separator \W+ is what makes the answer handle punctuation. A repetition separated only by a space is matched, and so is one separated by a comma or full stop, which is common in speech transcripts such as no, no. Because the separator cannot contain a letter or digit, the expression never joins two different words. The word boundaries prevent a match inside a longer word, so the the in thethe is not counted. By default the match is case-sensitive, so The the is not matched. Adding the i flag would match it, which may be desirable when capitalisation varies. Note that the expression also matches any immediately repeated word, including intentional repetition such as very very.

Algebraic equivalent

The set cannot be written with a back-reference in the algebraic notation, because the back-reference must remember an arbitrary word and repeat it. For a fixed, finite vocabulary of words, however, the set is regular: it is $\Sigma^*(w_1\ w_1 \mid w_2\ w_2 \mid \ldots \mid w_k\ w_k)\Sigma^*$ with the separator written explicitly, where Σ^* stands for any string. The Perl expression needs no vocabulary, and its back-reference is the feature that makes the answer general.

Common errors

- Using a fixed list of repeated words such as the the | a a | an an. This is correct only for that list and misses every other repetition.
- Writing \w+ \1 with a single space as the separator. This misses repetitions separated by punctuation, such as no, no, which the brief's instruction to deal with punctuation requires to be handled.
- Omitting the final word boundary. Then \b([A-Za-z]+)\W+\1 matches the first three letters of them after the, so the them is accepted although the two words differ. The closing \b is what requires the second word to be whole.

How the regular-expression engine matches

The following narrative traces the engine on two inputs, which shows why the expression behaves as described.

- On the input the the, the engine starts at position 0, where \b holds. The group ([A-Za-z]+) matches the greedy run the, and the separator \W+ consumes one space. The back-reference \1 must then match the text the at position 4, and it does. The final \b holds at the end of the string, so the engine reports a match in which group 1 is the.
- On the input the bug, the group again matches the, the separator matches the space, and \1 is tested against bug. The test fails. The engine backtracks: it shortens the group to th, then requires \W+ to match e, which fails; it then shortens the group to t, and the separator must match h, which also fails. No choice of group text at position 0 succeeds. The engine moves on, and the only other word starts are the and bug, neither of which is followed by a repeat. The overall result is no match.

3.5 Part (d): Each a Between Two b's

Question. The set of all strings from the alphabet {a, b} such that each a is immediately preceded and immediately followed by a b.

Answer

```
^(b+(ab+)*)?$
```

Explanation of symbols

Symbol	Explanation
`^` and `$`	Anchor the whole string, so the expression describes complete strings.
`b+`	The string opens with one or more b's. This ensures that no a appears at the start.
`(ab+)*`	Zero or more blocks, each consisting of an a followed by one or more b's. Every a is followed by a b and, because each block begins after a run of b's, is preceded by one.
`( … )?`	The whole body is optional, which admits the empty string and strings with no a at all.

Matching examples

String	Result
`bab`	Matches: the a is between two b's.
`bbabbb`	Matches: one a, surrounded by b's on both sides.
`babab`	Matches: two a's sharing the b between them; each a is preceded and followed by b.
`bbb`	Matches: contains no a, so the condition holds vacuously.

Non-matching examples

String	Reason rejected
`a`	The a is at the start and end, with no b before or after it.
`aba`	The first a is not preceded by a b.
`baab`	Two adjacent a's: the first a is followed by a, not a b.
`abb`	Begins with an a, which has no b before it.
`bba`	The final a is not followed by a b.

Punctuation and word boundaries

The empty string is included because the condition is universal: the empty string contains no a, so no a violates the condition. If the brief intended to exclude the empty string, the expression ^b+(ab+)*$ is used instead, which requires at least one b. Punctuation and word boundaries are not relevant here because the alphabet is restricted to the two symbols a and b and the question concerns individual characters, not words.

Algebraic equivalent

The algebraic form of the language is b+(ab+)* together with the empty string, which in the algebra is written (ε | b(b)*(ab(b)*)*) or more briefly (ε | b+(ab+)*), where ε stands for the empty string. This is the same set as the Perl expression, which uses ? to express the optional empty case and + to express one or more.

Common errors

- Writing (b|bab)*. This looks plausible, and it accepts bab and bbabbb, but it rejects babab. Each block in (b|bab)* contains its own b on both sides of the a, so two a's cannot share a single b between them. The correct form (ab+)* allows the shared b, because each block ends in a run of b's that the next block can begin after.

- Writing ^b+(ab+)*$ when the empty string is intended. This is a valid variant, and it is correct if the brief excludes the empty string. The optional group in the answer above includes it, following the vacuous reading of the condition.
- Forgetting the initial b+. Then (ab+)* accepts the string ab, in which the a has no b before it, which violates the condition that each a must be preceded by a b.

How the engine matches babab

The engine matches the first b with b+. It then matches ab at positions 1 and 2 with the first iteration of (ab+)*, and ab at positions 3 and 4 with the second. The end-of-string anchor holds, and the match succeeds. The outer group captures the whole string babab, and the inner group captures the last block, ab, because a repeated group keeps only its final iteration.

3.6 Part (e): Strings Starting with an Integer and Ending with a Word

Question. All strings that start at the beginning of the line with an integer and that end at the end of the line with a word.

Answer

```
^[+-]?[0-9]+\b.*\b[A-Za-z]+[.,;:!?]?$
```

Explanation of symbols

Symbol	Explanation
`^`	The line must begin with the integer.
`[+-]?`	An optional sign, so that -7 is an integer as well as 7.
`[0-9]+`	One or more digits forming the integer.
`\b`	A word boundary after the integer, so that the integer is a whole token: 42abc does not qualify.
`.*`	Any intervening characters, including none.
`\b[A-Za-z]+`	The final word, starting at a word boundary, so that ab3c does not end in the word c.
`[.,;:!?]?`	Optional terminal punctuation, which the brief's instruction to deal with punctuation requires.
`$`	The line must end immediately after the word (or its terminal punctuation).

Matching examples

String	Result
`1 apple`	Matches: integer 1 at the start; the word apple at the end.
`2024 is a year`	Matches: integer 2024; the line ends with the word year.
`-7 degrees`	Matches: the optional sign is part of the integer.
`3 cats.`	Matches: the word cats is followed by a full stop, which is permitted.

Non-matching examples

String	Reason rejected
`apples 3`	The line does not start with an integer.
`42`	No word at the end; the line ends with an integer.
`42abc`	No word boundary between the integer and the letters, so the integer is not a whole token.
`5 items 12`	The line ends with an integer rather than a word.

Punctuation and word boundaries

Two boundaries do the main work. The \b after the integer prevents the expression from splitting a token such as 42abc, which is not an integer followed by a word. The \b before the final word prevents a match in the middle of a token such as ab3c, where the final letters are preceded by a digit. Terminal punctuation is allowed as a deliberate choice: a line that ends in a sentence is still a line that ends with a word. If the brief is read strictly, so that the line must end on a letter, the optional group is removed and the expression becomes ^[+-]?[0-9]+\b.*\b[A-Za-z]+$, which would reject 3 cats.

Algebraic equivalent

The algebraic notation can describe this set only after the word boundary is written out as an explicit symbol class of non-word characters, combined with a digit class and a letter class. The result is long and hard to check, which is why the Perl form is preferred. The condition that the integer is a whole token is the part that is hardest to express in the algebra.

Common errors

- Dropping the boundary after the integer, as in ^[0-9]+.*[a-z]+$. This accepts 42abc, because the integer 42 is followed directly by letters and the rest of the line still ends in a word.

- Dropping the boundary before the final word. Then 5 ab3c is accepted, because the expression takes the final letter c as the last word, even though that letter belongs to the token ab3c.
- Requiring the line to end on a letter with no allowance for punctuation. The strict form ^[+-]?[0-9]+\b.*\b[A-Za-z]+$ rejects 3 cats., which ends with a full stop.

3.7 Part (f): Strings Containing Both grotto and raven

Question. All strings which have both the word grotto and the word raven in them, but not words such as grottos that merely contain the word grotto.

Answer

```
^(?=.*\bgrotto\b)(?=.*\braven\b)
```

Explanation of symbols

Symbol	Explanation
`^`	Anchors the start of the line, so the lookaheads are tested from the beginning.
`(?=…)`	A positive lookahead: the enclosed pattern must match at this position, but no characters are consumed.
`.*`	Any characters, so the word may appear anywhere in the line.
`\bgrotto\b`	The word grotto as a whole word. A boundary on each side means that grottos does not match.
`\braven\b`	The word raven as a whole word, tested by a second, independent lookahead.

The two lookaheads are independent, so the expression does not depend on the order in which the words occur. A line in which raven comes before grotto is accepted in the same way as one in which grotto comes first. If the matched line itself is required, the expression can be extended to ^(?=.*\bgrotto\b)(?=.*\braven\b).*$. An equivalent alternation without lookahead is \bgrotto\b.*\braven\b|\braven\b.*\bgrotto\b, but this needs two alternatives to cover both orders, which is why the lookahead form is preferred.

Matching examples

String	Result
`the grotto and the raven`	Matches: both words are present as whole words.
`raven flew into the grotto`	Matches: raven appears first, which the lookahead form accepts.
`grotto, raven`	Matches: the comma after grotto is a boundary, so the word is recognised.

Non-matching examples

String	Reason rejected
`grottos and raven`	grottos is a different word; the \b after grotto fails because s follows.
`the raven`	The word grotto is absent.
`ravens near the grotto`	ravens is a different word; the \b after raven fails.

Punctuation and word boundaries

The boundaries \b test for a transition from a word character to a non-word character, so punctuation such as a comma, full stop or apostrophe acts as a word end. A possessive such as grotto's therefore matches the word grotto. This is a reasonable reading of the brief, since the apostrophe is punctuation, but a stricter reading could exclude it. Case-sensitive matching is the default; the i flag would also match Grotto and Raven.

Algebraic equivalent

The algebraic notation has no notion of a word boundary, so the boundary must be written as an explicit separator class. Over the alphabet of all characters, the language can be described with Σ* for any string, a separator class for the non-letter characters, and the two words. Two alternatives are needed, one for each order of the words, because the algebra has no conjunction. The Perl form avoids both problems: each lookahead tests one word anywhere in the line, and the two tests are combined by ordinary conjunction.

Common errors

- Writing a single order such as grotto.*raven. This is order-dependent and rejects raven flew into the grotto, in which raven comes first.
- Omitting the word boundaries. Then grotto matches inside grottos, which the brief specifically excludes.
- Testing for the words at a single fixed position, for example with ^grotto. The words may occur anywhere in the line, so the lookahead must be allowed to search forward with .*.

Matching examples in more detail

The lookahead form matches each line once, at position 0, because it consumes no characters. If the matched text itself is required, the trailing .*$ in the extended expression is used, so

that the whole line is returned as the match. Lines that contain the two words in either order, with any amount of other text between them, are all returned in full by the extended form.

3.8 Part (g): Capturing the First Word of a Sentence

Question. Write a pattern which places the first word of an English sentence in a register.

Answer

```
(?:^|[.!?]\s+)\W*([A-Za-z]+)
```

In Perl, a successful match stores the captured text in the numbered variable $1. This is the register: the first word of the sentence is captured by the group ([A-Za-z]+), and the program reads it from $1 after the match. The pattern is applied to each sentence in turn, and in a global search every sentence-initial word is captured in sequence.

Explanation of symbols

Symbol	Explanation
`(?:^|[.!?]\s+)`	A non-capturing group that marks where a sentence begins: either the start of the line, or a sentence-ending mark followed by white space.
`\W*`	Skips any leading punctuation or white space, such as opening quotation marks or brackets, so that the word after an opening quotation mark is captured.
`([A-Za-z]+)`	The register: capturing group 1 holds the first word of the sentence.

Matching examples

String	Captured register (\$1)
`The cat sat. Then it slept.`	The, then Then (two sentences, two captures)
`Rain fell.`	Rain
`Yes! Maybe later.`	Yes, then Maybe

Non-matching examples

String	Reason rejected
`42 dogs ran.`	The sentence begins with a digit, which \W* cannot skip and [A-Za-z] cannot match.
`!!! 2 words`	After the punctuation is skipped, the next character is a digit, so no word is captured.
`` ` (empty string ``	No sentence start and no letters exist, so there is nothing to capture.

Punctuation and word boundaries

Punctuation is handled in two places. Sentence ends are recognised by the characters ., ! and ? followed by white space, and leading punctuation inside the sentence is skipped by \W*. The pattern has a known limitation: a full stop inside an abbreviation, such as Dr. Smith, makes the pattern treat Smith as the first word of a new sentence. A production system would need an abbreviation list. The pattern also captures only letters, so a sentence starting with a contraction such as Don't is register Don. Extending the capture to ([A-Za-z][A-Za-z'-]*) would keep the whole contraction.

Algebraic equivalent

Capturing is not a property of a language. The algebraic notation can describe the set of strings that begin with a given word, but a register that holds whichever word occurs first has no algebraic counterpart. This part therefore relies on a feature of Perl, the numbered capture variable $1, which the algebraic notation does not provide.

Common errors

- Omitting the sentence anchor and capturing every word. A pattern such as ([A-Za-z]+) with no context captures the first word of the string and then, in a global search, every later word as well.
- Using only ^ as the anchor. This captures the first word of the line but misses the first words of later sentences in the same line, such as Then in The cat sat. Then it slept.
- Treating every full stop as a sentence end. Abbreviations break this: Dr. Smith is read as two sentences, and Smith is captured. The pattern tested on Dr. Smith went home. returns Dr and Smith, which is the limitation noted above.
- Forgetting the leading \W*. A sentence that opens with a quotation mark, such as "Yes", would not be captured, because the quotation mark is not a letter.

3.9 Summary of Question 2 Results

Table 11a summarises the seven answers. Each expression was executed against its matching and non-matching examples, and the examples listed in each part are the ones that were run. All seven expressions gave the expected result on every example.

Table 11a: Summary of the seven regular expressions and the tests applied

Part	Language	Matching tested	Non-matching tested	Result
(a)	All alphabetic strings	4	5 (including the empty string)	All as expected
(b)	Lowercase strings ending in b	4	5	All as expected
(c)	Consecutive repeated words	4	4	All as expected
(d)	Each a between two b's	4	5	All as expected
(e)	Integer at start, word at end	4	4	All as expected
(f)	Both grotto and raven present	3	3	All as expected
(g)	First word of a sentence	3	3	All as expected

4. Question 3: Minimum Edit Distance

Question. Computing minimum edit distances by hand, figure out whether DRIVE is closer to BRIEF or to DIVERS, and what the edit distance is. You may use any version of distance that you like. (10 marks; CO1; BLT4)

4.1 Definition of Minimum Edit Distance

The **minimum edit distance** between two strings is the smallest total cost of a sequence of edit operations that transforms the first string into the second. It is a measure of dissimilarity: two strings that are close need few edits, and two strings that are far apart need many. The measure underlies spelling correction, the evaluation of speech recognition output and the alignment of biological sequences. The standard algorithm for computing it was described by Levenshtein (1966) and was given its dynamic-programming form by Wagner and Fischer (1974).

An **alignment** is a way of writing the two strings one above the other, so that each character of one string is either matched with a character of the other, substituted for it, or paired with a gap that represents an insertion or a deletion. The cost of the alignment is the sum of the costs of its columns, and the minimum edit distance is the smallest such cost over all alignments.

4.2 Allowed Operations and Costs

Three operations are allowed: **insertion** of a character into the first string, **deletion** of a character from it, and **substitution** of one character by another. A character that already matches costs nothing. The cost of each operation depends on the version of the distance chosen. The brief permits any version, so two standard versions are used and compared in this chapter. Both use the same insertion and deletion cost of 1. They differ in the cost of a substitution.

Table 12: Edit operations and their costs under the two versions used

Operation	Description	Cost (Version A: sub = 2)	Cost (Version B: sub = 1)
Match	Identical characters aligned	0	0
Insertion	Insert a character from the second string	1	1
Deletion	Delete a character from the first string	1	1
Substitution	Replace one character by a different one	2	1

Version A, with substitution costing 2, is the Levenshtein formulation that treats a substitution as one deletion followed by one insertion. It is the version in which a substitution is no cheaper than the deletion and insertion that would replace it, and it is a widely used textbook convention. Version B, with substitution costing 1, is the unit-cost Levenshtein distance. Version A is used as the primary result in this chapter, and Version B is reported in Section 4.6 so that the sensitivity of the answer to this choice is visible.

4.3 Dynamic Programming Formulation

Let s be the first string of length n and t the second string of length m. Let D(i, j) be the minimum edit distance between the first i characters of s and the first j characters of t. The base cases are D(i, 0) = i, since i deletions are needed to empty the prefix of s, and D(0, j) = j, since j insertions are needed to build the prefix of t. For i and j both at least 1, the recurrence is:

```
D(i, j) = min of:
    D(i−1, j) + 1              deletion
    D(i, j−1) + 1              insertion
    D(i−1, j−1) + c(sᵢ, tⱼ)    match or substitution

where c = 0 if sᵢ = tⱼ, and c = sub cost otherwise.
```

The final answer is D(n, m). Each cell is computed from three neighbours, and the tables in the next sections are filled in this order, row by row. To recover the alignment, the computation is traced backwards from D(n, m): at each cell the trace moves to the neighbour that produced the minimum. The alignment is read off from the sequence of moves, and its cost equals D(n, m) by construction.

4.4 Computing DRIVE to BRIEF

The first string is DRIVE (n = 5) and the second is BRIEF (m = 5). Table 13 gives the complete dynamic-programming table under Version A. Row labels are the characters of DRIVE and column labels are the characters of BRIEF, with ε for the empty prefix.

Table 13: Dynamic-programming table for DRIVE → BRIEF, Version A (sub = 2)

DRIVE \ BRIEF	ε	B	R	I	E	F
ε	0	1	2	3	4	5
D	1	2	3	4	5	6
R	2	3	2	3	4	5
I	3	4	3	2	3	4
V	4	5	4	3	4	5
E	5	6	5	4	3	**4**

The bottom-right cell gives the answer, D(5, 5) = 4. Several cells show how the costs accumulate. The cell for D against B, at row 1 and column 1, is 2: the substitution of D by B costs 2, which is no cheaper than a deletion followed by an insertion. The cell for R against R, at row 2 and column 2, is also 2, because the R's match at no cost and the prefix before them costs 2. The cell for E against E, at row 5 and column 4, is 3, which is the cheapest way to align DRIVE with BRIE. The final cell is one insertion (the F) beyond that cell, so D(5, 5) = D(5, 4) + 1 = 3 + 1 = 4.

The backtrace makes the optimal path explicit. From D(5, 5) = 4 the last move is the insertion of F, from D(5, 4) = 3. D(5, 4) is reached by matching E with E, from D(4, 3) = 2. D(4, 3) is reached by deleting V, from D(3, 3) = 1. D(3, 3) is reached by matching I with I, from D(2, 2) = 2. D(2, 2) is reached by matching R with R, from D(1, 1) = 2. Finally, D(1, 1) is reached by substituting D with B, from D(0, 0) = 0. The alignment therefore contains one substitution (D with B, cost 2), three matches (R, I and E, cost 0), one deletion (V, cost 1) and one insertion (F, cost 1). Its cost is 2 + 0 + 0 + 1 + 0 + 1 = 4, which agrees with the table.

Alignment table

Table 14: Optimal alignment of DRIVE and BRIEF under Version A

Source (DRIVE)	Target (BRIEF)	Operation	Cost
D	B	Substitution	2
R	R	Match	0
I	I	Match	0
V	–	Deletion	1
E	E	Match	0
–	F	Insertion	1
Total			**4**

Step-by-step transformation

Applying the alignment to DRIVE produces BRIEF in three steps:

- Step 1 (substitution, cost 2): replace the initial D with B, giving **BRIVE**.
- Step 2 (deletion, cost 1): remove the V, giving **BRIE**.
- Step 3 (insertion, cost 1): add the F at the end, giving **BRIEF**.

The total cost is 4. Version B gives a different alignment of the same length, with three substitutions in place of the substitution and deletion above, and its cost is 3 (Section 4.6).

4.5 Computing DRIVE to DIVERS

The second comparison is between DRIVE (n = 5) and DIVERS (m = 6). Table 15 gives the dynamic-programming table under Version A.

Table 15: Dynamic-programming table for DRIVE → DIVERS, Version A (sub = 2)

DRIVE \ DIVERS	ε	D	I	V	E	R	S
ε	0	1	2	3	4	5	6
D	1	**0**	1	2	3	4	5
R	2	1	2	3	4	3	4
I	3	2	1	2	3	4	5
V	4	3	2	1	2	3	4
E	5	4	3	2	1	2	3

The answer is D(5, 6) = 3. The first row and column are the base cases. The first diagonal match, D with D at cell (1, 1), has value 0, so the two strings agree in their first letter at no cost. The cell for E against E, at row 5 and column 4, holds the value 1: the D, I, V and E of DRIVE are matched in turn at no cost except for the single deletion of R. The bottom-right cell then adds the two trailing letters of DIVERS, R and S, each at cost 1, giving D(5, 6) = 1 + 1 + 1 = 3, which is one deletion of R from DRIVE and two insertions, of R and S, at the end of DIVERS.

Alignment table

Table 16: Optimal alignment of DRIVE and DIVERS under Version A

Source (DRIVE)	Target (DIVERS)	Operation	Cost
D	D	Match	0
R	–	Deletion	1
I	I	Match	0
V	V	Match	0
E	E	Match	0
–	R	Insertion	1
–	S	Insertion	1
Total			**3**

Step-by-step transformation

- Step 1 (deletion, cost 1): remove the R of DRIVE, giving **DIVE**.
- Step 2 (insertion, cost 1): append R, giving **DIVER**.
- Step 3 (insertion, cost 1): append S, giving **DIVERS**.

The total cost is 3. No substitution is needed, because the four letters D, I, V and E occur in both words in the same order. The shared subsequence DIVE, of length four, is the longest common subsequence of the two words, and the edit cost of 3 is consistent with that: with insertion and deletion alone, the cost equals the length of the first string plus the length of the second minus twice the length of their longest common subsequence, which is $5 + 6 - 2 \times 4 = 3$.

4.6 Sensitivity to the Cost Model: Version B

Under Version B, with substitution costing 1, the same two comparisons give different tables. Tables 17 and 18 give the results. The DRIVE to DIVERS table does not change in its final value, because that alignment contains no substitution. The DRIVE to BRIEF table changes, and its final value falls from 4 to 3.

Table 17: Dynamic-programming table for DRIVE → BRIEF, Version B (sub = 1)

DRIVE \ BRIEF	ε	B	R	I	E	F
ε	0	1	2	3	4	5
D	1	1	2	3	4	5
R	2	2	1	2	3	4
I	3	3	2	1	2	3
V	4	4	3	2	2	3
E	5	5	4	3	2	**3**

Table 18: Dynamic-programming table for DRIVE → DIVERS, Version B (sub = 1)

DRIVE \ DIVERS	ε	D	I	V	E	R	S
ε	0	1	2	3	4	5	6
D	1	**0**	1	2	3	4	5
R	2	1	1	2	3	3	4
I	3	2	1	2	3	4	4
V	4	3	2	1	2	3	4
E	5	4	3	2	1	2	3

Under Version B the optimal alignment of DRIVE and BRIEF is D→B, R=R, I=I, V→E, E→F, with three substitutions and cost 3. An equally cheap alternative deletes V and inserts F, as in Version A, at cost 3 as well. The two results give the same distance, 3, for both words, so under unit-cost substitution the comparison is a tie.

4.7 Final Comparison

Table 19 brings the results together. The comparison is made under each version, and the closer word is identified.

Table 19: Final comparison of edit distances

Cost version	DRIVE → BRIEF	DRIVE → DIVERS	Closer word
Version A (ins = del = 1, sub = 2)	4	**3**	**DIVERS**
Version B (ins = del = 1, sub = 1)	3	3	Tie

4.8 Justified Final Answer

DRIVE is closer to DIVERS than to BRIEF. Under the primary cost model, Version A, the minimum edit distance is **3** for DRIVE → DIVERS and **4** for DRIVE → BRIEF. The edit distance from DRIVE to DIVERS is 3, which is smaller than the distance to BRIEF, so DIVERS is the closer word.

The justification rests on the shared letters of the two pairs. DRIVE and DIVERS share the subsequence DIVE of four letters, which the optimal alignment matches at no cost, so the only edits are one deletion and two insertions: $5 + 6 - 2 \times 4 = 3$. DRIVE and BRIEF share the subsequence RIE of three letters, and insertion and deletion alone would therefore cost $5 + 5 - 2 \times 3 = 4$. Substitution does not reduce this, because under Version A replacing D by B costs as much as deleting D and inserting B. The result is that BRIEF is at distance 4 and DIVERS at distance 3, so DIVERS is the closer word.

The answer is conditional on the cost model. Under Version B, the unit-cost Levenshtein distance, both words are at distance 3, and the comparison does not separate them. The answer given above therefore depends on Version A, which is the model that counts a substitution as one deletion plus one insertion. This is the standard choice in many NLP texts, and it is the one that gives a strict answer to the question asked. The tie under Version B is reported rather than hidden, because a reader who prefers unit substitution cost should know that the conclusion does not hold for that model.

4.9 Verification of the Calculations

Each table was computed twice. The dynamic-programming values were produced by iterating the recurrence over the cells of the table, and the same distances were recomputed by an independent memoised recursive implementation of the same definition. For both words and both cost versions, the two methods agreed: 4 and 3 under Version A, and 3 and 3 under Version B. The alignment costs were also summed column by column and compared with the table value, and they agreed in every case. Each transformation in Sections 4.4 and 4.5 was applied to the source string and checked to produce the target string exactly.

4.10 Enumeration of All Optimal Alignments

The alignments in Tables 14 and 16 are not the only optimal ones. A complete backtrace from the bottom-right cell, following every predecessor that attains the minimum, enumerates all optimal alignments. Table 14a lists the results for each pair under each cost version that

distinguishes the cases. Operations are written from left to right: a letter pair such as D→B is a substitution, a letter pair such as E=E is a match, and del or ins denote a deletion or an insertion from the source string or into it.

Table 14a: All optimal alignments found by complete backtrace

Pair	Cost version	Number of optimal alignments	Alignments (operations left to right)
DRIVE → BRIEF	A (sub = 2)	3	(1) D→B, R=R, I=I, del V, E=E, ins F (2) ins B, del D, R=R, I=I, del V, E=E, ins F (3) del D, ins B, R=R, I=I, del V, E=E, ins F
DRIVE → BRIEF	B (sub = 1)	2	(1) D→B, R=R, I=I, V→E, E→F (2) D→B, R=R, I=I, del V, E=E, ins F
DRIVE → DIVERS	A (sub = 2)	1	D=D, del R, I=I, V=V, E=E, ins R, ins S
DRIVE → DIVERS	B (sub = 1)	1	D=D, del R, I=I, V=V, E=E, ins R, ins S

Two observations follow. For DRIVE and BRIEF under Version A, the first alignment replaces D by B with one substitution, while the other two replace it through a deletion and an insertion, in either order. Because a substitution costs 2 under Version A, which is the cost of a deletion plus an insertion, these alternatives all cost 4. Under Version B the two alignments are genuinely different in cost structure, but both cost 3. The second observation is that DRIVE and DIVERS have a unique optimal alignment under both versions. The closeness of DRIVE to DIVERS therefore does not depend on which of several alignments is chosen, which strengthens the conclusion of Section 4.8.

4.11 A Damerau–Levenshtein Variant

The brief allows any version of distance, so a third family is also considered. The Damerau–Levenshtein distance adds one further operation: the transposition of two adjacent characters, which costs 1. Transposition is a common typing error, which is why it is included in many spelling-correction systems. The recurrence gains one extra term. If the two characters at positions i − 1 and i of the source are the reverse of the two characters at positions j − 1 and j of the target, then D(i, j) may also take the value D(i − 2, j − 2) + 1.

Neither pair benefits from a transposition under either substitution cost. For DRIVE and BRIEF, the pairs of adjacent characters in DRIVE are DR, RI, IV and VE, and the reversed pairs RD, IR, VI and EV do not produce a cheaper alignment with BRIEF. For DRIVE and DIVERS, the reversal of RI would give IR, which is not present in the target at the matching

position. Table 14b gives all four cost versions, which combine substitution cost 2 or 1 with transposition allowed or not.

Table 14b: Minimum edit distances under four cost versions

Version	Operations and costs	DRIVE → BRIEF	DRIVE → DIVERS	Closer word
A	ins = del = 1, sub = 2	4	3	DIVERS
B	ins = del = 1, sub = 1	3	3	Tie
C	ins = del = 1, sub = 2, transposition = 1	4	3	DIVERS
D	ins = del = 1, sub = 1, transposition = 1	3	3	Tie

Versions A and C agree, and versions B and D agree. The conclusion is therefore determined by the substitution cost alone, and transposition does not change it for these two word pairs. Whether a substitution counts as one edit or two is the only modelling choice that matters here. This is the reason the report presents Version A as the primary answer, and why Version B is reported in full rather than omitted.

4.12 Complexity and Implementation

The dynamic-programming table for strings of lengths n and m has (n + 1)(m + 1) cells, and each cell is computed from three neighbours in constant time. The algorithm therefore runs in O(nm) time. Keeping the full table, which is needed to recover the alignment, also takes O(nm) space. For DRIVE and BRIEF the table has 6 × 6 = 36 cells, and for DRIVE and DIVERS it has 6 × 7 = 42 cells. These are small enough to fill by hand, which is the purpose of the exercise.

Only the previous row is needed to compute the distance itself, so the space can be reduced to O(min(n, m)) by keeping two rows and making the shorter string the row length. The alignment cannot be recovered from the distance alone, however, so a practical system stores either the full table or a small code for the predecessor of each cell. Appendix B gives a Python implementation that computes the full table, with a parameter for the cost of each operation, so that Versions A and B can be reproduced by changing one argument.

4.13 Applications in Natural Language Processing

Minimum edit distance has several practical uses in natural language processing. In spelling correction, candidate corrections lying within a small edit distance of a misspelled word are

ranked by distance, often combined with word frequency or a language-model score. Distance is also used to filter candidates quickly before a more expensive comparison is made.

In speech recognition, the standard evaluation measure is the word error rate. It is defined as the word-level edit distance between the reference transcript and the recognised hypothesis, divided by the number of words in the reference: WER = (S + D + I) / N, where S, D and I are the numbers of substitutions, deletions and insertions in the minimum-cost alignment, and N is the number of reference words. This is the same dynamic programme as in this chapter, applied to sequences of words rather than characters, and with unit costs, which is Version B in the terminology above.

The same recurrence underlies sequence alignment in computational biology, where substitution costs are taken from a scoring matrix rather than fixed by hand, and it appears in the alignment of translations and transcripts in corpus processing. The character-level measure studied in this chapter is therefore a small, complete example of a method that is used at much larger scale.

4.14 Cell-by-Cell Working Under Version A

For readers who wish to check the tables of Sections 4.4 and 4.5, Tables 14c and 14d show every cell of the computation. Each row gives the three candidate values for one cell: the deletion from the cell above, the insertion from the cell to the left, and the match or substitution from the diagonal cell. The cell takes the smallest of the three. Cells are listed row by row, so the order of computation is the order of the rows. Every cell depends only on cells that appear earlier in the list, so the table can be filled in one pass.

Table 14c: Cell-by-cell computation for DRIVE → BRIEF (Version A)

Cell (i, j)	Characters	Deletion D(i−1,j)+1	Insertion D(i,j−1)+1	Match / sub D(i−1,j−1)+c	D(i, j)
(1, 1)	D / B	2	2	2 (sub)	**2**
(1, 2)	D / R	3	3	3 (sub)	**3**
(1, 3)	D / I	4	4	4 (sub)	**4**
(1, 4)	D / E	5	5	5 (sub)	**5**
(1, 5)	D / F	6	6	6 (sub)	**6**
(2, 1)	R / B	3	3	3 (sub)	**3**
(2, 2)	R / R	4	4	2 (match)	**2**
(2, 3)	R / I	5	3	5 (sub)	**3**
(2, 4)	R / E	6	4	6 (sub)	**4**

Cell (i, j)	Characters	Deletion D(i−1,j)+1	Insertion D(i,j−1)+1	Match / sub D(i−1,j−1)+c	D(i, j)
(2, 5)	R / F	7	5	7 (sub)	**5**
(3, 1)	I / B	4	4	4 (sub)	**4**
(3, 2)	I / R	3	5	5 (sub)	**3**
(3, 3)	I / I	4	4	2 (match)	**2**
(3, 4)	I / E	5	3	5 (sub)	**3**
(3, 5)	I / F	6	4	6 (sub)	**4**
(4, 1)	V / B	5	5	5 (sub)	**5**
(4, 2)	V / R	4	6	6 (sub)	**4**
(4, 3)	V / I	3	5	5 (sub)	**3**
(4, 4)	V / E	4	4	4 (sub)	**4**
(4, 5)	V / F	5	5	5 (sub)	**5**
(5, 1)	E / B	6	6	6 (sub)	**6**
(5, 2)	E / R	5	7	7 (sub)	**5**
(5, 3)	E / I	4	6	6 (sub)	**4**
(5, 4)	E / E	5	5	3 (match)	**3**
(5, 5)	E / F	6	4	6 (sub)	**4**

Table 14d: Cell-by-cell computation for DRIVE → DIVERS (Version A)

Cell (i, j)	Characters	Deletion D(i−1,j)+1	Insertion D(i,j−1)+1	Match / sub D(i−1,j−1)+c	D(i, j)
(1, 1)	D / D	2	2	0 (match)	**0**
(1, 2)	D / I	3	1	3 (sub)	**1**
(1, 3)	D / V	4	2	4 (sub)	**2**
(1, 4)	D / E	5	3	5 (sub)	**3**
(1, 5)	D / R	6	4	6 (sub)	**4**
(1, 6)	D / S	7	5	7 (sub)	**5**
(2, 1)	R / D	1	3	3 (sub)	**1**
(2, 2)	R / I	2	2	2 (sub)	**2**
(2, 3)	R / V	3	3	3 (sub)	**3**
(2, 4)	R / E	4	4	4 (sub)	**4**
(2, 5)	R / R	5	5	3 (match)	**3**
(2, 6)	R / S	6	4	6 (sub)	**4**
(3, 1)	I / D	2	4	4 (sub)	**2**
(3, 2)	I / I	3	3	1 (match)	**1**
(3, 3)	I / V	4	2	4 (sub)	**2**
(3, 4)	I / E	5	3	5 (sub)	**3**

Cell (i, j)	Characters	Deletion D(i−1,j)+1	Insertion D(i,j−1)+1	Match / sub D(i−1,j−1)+c	D(i, j)
(3, 5)	I / R	4	4	6 (sub)	**4**
(3, 6)	I / S	5	5	5 (sub)	**5**
(4, 1)	V / D	3	5	5 (sub)	**3**
(4, 2)	V / I	2	4	4 (sub)	**2**
(4, 3)	V / V	3	3	1 (match)	**1**
(4, 4)	V / E	4	2	4 (sub)	**2**
(4, 5)	V / R	5	3	5 (sub)	**3**
(4, 6)	V / S	6	4	6 (sub)	**4**
(5, 1)	E / D	4	6	6 (sub)	**4**
(5, 2)	E / I	3	5	5 (sub)	**3**
(5, 3)	E / V	2	4	4 (sub)	**2**
(5, 4)	E / E	3	3	1 (match)	**1**
(5, 5)	E / R	4	2	4 (sub)	**2**
(5, 6)	E / S	5	3	5 (sub)	**3**

The final rows of both tables agree with the bottom-right entries of Tables 13 and 15, which are 4 for BRIEF and 3 for DIVERS. Tracing back from those cells through the entries that produced each minimum recovers the alignments of Tables 14 and 16.

5. Conclusion

This report answered the three questions of the assignment. For the first question, a deterministic finite-state automaton with twelve states and five accepting states was designed to recognise absolute date expressions. It covers month-first, ordinal-of-month, day-first and holiday forms, with an optional year, and it rejects deictic and malformed expressions. Its transition table was verified against an independent regular expression over all token strings up to length six, and the two agreed on every string.

For the second question, seven regular expressions in Perl notation were given, one for each part (a) to (g). Each was tested against matching and non-matching examples, and each handles punctuation and word boundaries explicitly. The expressions illustrate the main constructs of regular expressions: anchors and character classes, optional and repeated elements, back-references, lookahead and capturing. Where the brief admits more than one reading, the chosen reading is stated, and the alternative is noted.

For the third question, the minimum edit distances from DRIVE to BRIEF and to DIVERS were computed by dynamic programming and verified. Under the primary cost model, substitution costing 2, DRIVE is closer to DIVERS, at distance 3, than to BRIEF, at distance 4. Under unit-cost substitution the two distances are equal at 3, and the answer depends on the chosen cost model. The report states this dependence rather than presenting a single unqualified result.

5.1 Limitations and Further Work

The automaton recognises a bounded set of expressions. Its correctness is established against the grammar it was designed from, and that agreement was checked exhaustively over all token strings of length up to six. Longer inputs were not checked exhaustively, although the structure of the automaton, in which the year suffix is shared and every state has a single purpose, makes the general argument straightforward. Extending the word classes to abbreviations or to other calendars would require the same kind of check to be repeated.

The regular expressions were tested on the examples given in each part, and each was reasoned about construct by construct. A stronger check would run each expression on a large corpus of text, using a real Perl interpreter, and compare the captured words with a hand-labelled reference. Such a corpus would expose the abbreviation and contraction problems noted in part (g), and would give a measure of how often the punctuation policy and the word boundaries behave as intended.

The edit-distance answer depends on the cost model, as Section 4.7 showed. A future report should state the cost model in the first sentence of the answer. A natural extension is a weighted distance in which the cost of a substitution depends on the characters involved, for example on their keyboard proximity, which would reflect typing errors more closely than a single fixed cost.

Appendix A: Implementation of the Date Automaton

The listing below is the program used to check the automaton in Section 2.14. It stores the transition table of Table 3 as a dictionary, classifies each token, and follows the arcs. Running the file prints the decision for each example.

```
import re

MONTHS = ["January", "February", "March", "April", "May", "June", "July",
          "August", "September", "October", "November", "December"]
HOLIDAYS = {"Christmas", "Christmas Eve", "Christmas Day", "New Year's Day",
            "Valentine's Day", "Halloween", "Independence Day"}

def ordinal(n):
    if 10 <= n % 100 <= 20:
        suffix = "th"
    else:
        suffix = {1: "st", 2: "nd", 3: "rd"}.get(n % 10, "th")
    return f"{n}{suffix}"

CARDS = {str(n) for n in range(1, 32)}
ORDS = {ordinal(n) for n in range(1, 32)}

def classify(tok):
    if tok in MONTHS:  return "MONTH"
    if tok in CARDS:   return "CARD"
    if tok in ORDS:    return "ORD"
    if tok == "the": return "THE"
    if tok == "of": return "OF"
    if tok == ",": return "COMMA"
    if re.fullmatch(r"(19|20)\d\d", tok): return "YEAR"
    if tok in HOLIDAYS: return "HOL"
    return "OTHER"

# delta[state][class] = next state; accepting states:
DELTA = {
    "q0":  {"MONTH": "q1", "CARD": "q9", "ORD": "q9", "THE": "q5", "HOL": "q11"},
    "q1":  {"CARD": "q2", "ORD": "q2", "YEAR": "q4"},
    "q2":  {"COMMA": "q3", "YEAR": "q4"},
    "q3":  {"YEAR": "q4"},
    "q4":  {},
    "q5":  {"ORD": "q6"},
    "q6":  {"OF": "q7"},
    "q7":  {"MONTH": "q8"},
    "q8":  {"COMMA": "q3", "YEAR": "q4"},
    "q9":  {"MONTH": "q10"},
    "q10": {"COMMA": "q3", "YEAR": "q4"},
    "q11": {"COMMA": "q3"},
}
ACCEPTING = {"q2", "q4", "q8", "q10", "q11"}

def accepts(tokens):
    state = "q0"
    for tok in tokens:
        cls = classify(tok)
        if cls not in DELTA[state]:
            return False          # no arc: reject immediately
        state = DELTA[state][cls]
    return state in ACCEPTING

if __name__ == "__main__":
    tests = ["March 15", "the 22nd of November", "Christmas", "March 15, 2026",
             "the 22nd November", "yesterday"]
    for s in tests:
        toks = re.findall(r"New Year's Day|Christmas Eve|Christmas Day|Valentine's Day|"
                          r"Independence Day|[A-Za-z]+|\d+(?:st|nd|rd|th)?|,", s)
        print(f"{s!r:28} -> {'ACCEPT' if accepts(toks) else 'reject'}")
```

Appendix B: Implementation of the Minimum Edit Distance

The listing below computes the dynamic-programming table of Section 4.3. The operation costs are parameters, so Versions A and B are reproduced by changing the substitution cost. Running the file prints the distances used in Section 4.7.

```
def min_edit_distance(source, target, sub_cost=2, ins_cost=1, del_cost=1):
    """Return the minimum edit distance and the full DP table."""
    n, m = len(source), len(target)
    D = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        D[i][0] = i * del_cost
    for j in range(1, m + 1):
        D[0][j] = j * ins_cost
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            same = source[i - 1] == target[j - 1]
            D[i][j] = min(
                D[i - 1][j] + del_cost,                              # deletion
                D[i][j - 1] + ins_cost,                             # insertion
                D[i - 1][j - 1] + (0 if same else sub_cost),        # match /
substitution
            )
    return D[n][m], D

if __name__ == "__main__":
    for target in ("BRIEF", "DIVERS"):
        for sub in (2, 1):
            d, _ = min_edit_distance("DRIVE", target, sub_cost=sub)
            print(f"DRIVE -> {target:<6} sub={sub}: {d}")
```

Appendix C: Glossary of Key Terms

The terms below are used throughout the report. Each definition is given in the sense in which the term is used in this chapter.

Term	Meaning in this report
Absolute date	A date expression whose reference does not depend on the time of utterance, such as March 15.
Accepting state	A state of an automaton in which a run ends if the input is a member of its language.
Alignment	A pairing of the characters of two strings, with each pair treated as a match, substitution, insertion or deletion.
Alphabet (Σ)	The finite set of input symbols an automaton reads. Here, the word classes of Table 1.
Back-reference	A regular-expression construct, \1 in Perl, that matches the same text as an earlier capturing group.
Class (word class)	A named set of words that behave identically in the grammar, used as an arc label to save arcs.
Deictic expression	A time expression whose reference depends on the context of utterance, such as yesterday.
Deterministic automaton	An automaton in which each state has at most one successor for each input symbol.
Dynamic programming	A method that solves a problem by filling a table of subproblem solutions, each computed once from earlier entries.
Edit distance	The minimum total cost of insertions, deletions and substitutions that transforms one string into another.
Lookahead	A zero-width regular-expression construct that tests whether a pattern matches at the current position, without consuming input.
Regular language	A set of strings that some finite automaton accepts, equivalently one described by a regular expression.
Transition function (δ)	The rule that gives the next state for each state and input symbol.
Word boundary (\b)	A zero-width position between a word character and a non-word character, or at the edge of the string.